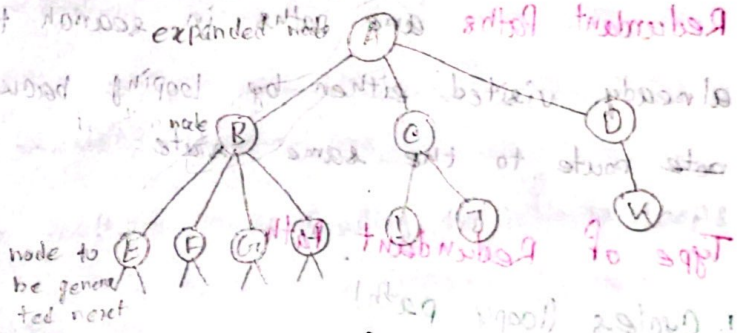
AI Search.pdf

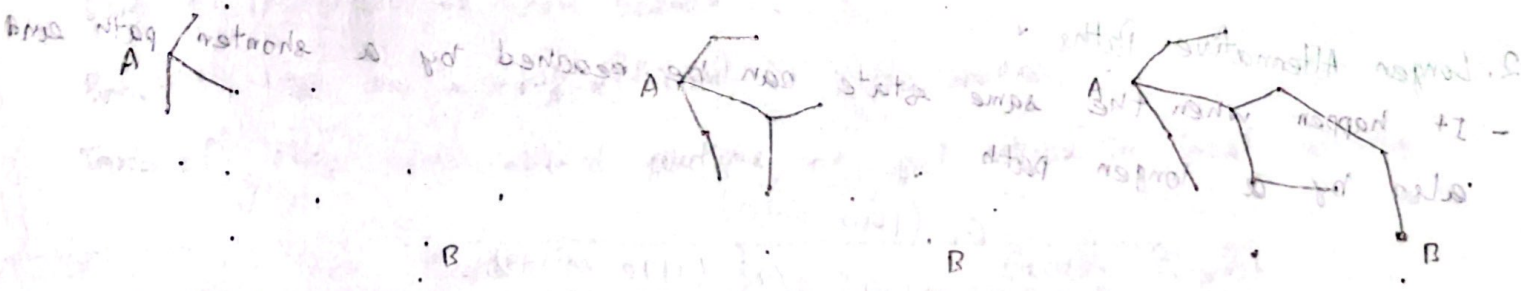
A **Search Algorithm** is a step by step procedure used to explore a problem's state space in order to find a sequence of actions that leads from an inital state to a goal state, or to determine that no such solution exists.

1. Tree Search

~ is a search method that explores possible solutions by expanding nodes in a tree, ~~starts~~ staring from the initial state and continuing until the goal state is found.

- Represents the problem as a tree
- Root node = initial state
- Edges = actions leading to new state
- Expands nodes to generate new child nodes, called successors
- Stops when it reaches the gol state or the tree is fully expanded
- Does not check for repeated states → can cause cycle or redundant exploration





Type of queues in Search Algorithm

1. Priority Queue

- Pops nodes with lowest cost base on evaluation function f(n)
- Use in Best-First search, Uniform cost search, A*
- Heuristic-driven, goal-focused

2. FIFO Queue (First in First Out)

- Pop the node that was added earliest
- Used in Breadth-First Search (BFS)
- Explores level by level

3. LIFO Queue (Last In First Out / Stack)

- Pop the most recently added node
- Use in Depth First Search (DFS)
- Explores deep paths first

Scanned with CamScanner

Properties of Search Algorithms

Completeness: An algorithm is complete if it is guaranteed to find a solution when one exists

Optimality: An algorithm is optimal if it always finds the best solution (lowest cost)

Time Complexity: How long does it take to find the solution

Space Complexity: How much memory does it need during the search

Redundant Paths are paths in search tree or graph that reach a state already visited, either by looping back (cycle) or by taking a longer unnecessary route to the same state

Type of Redundant Path

1. Cycles (loopy path)

- A cycle is a path in a search tree or graph that starts from a state, goes through a sequence of actions and then returns back to the same state.

$$A \rightarrow B \rightarrow C \rightarrow A$$

2. Longer Alternative Paths

- It happen when the same state can be reached by a shorter path and also by a longer path

$A \rightarrow B \rightarrow Gr$ (140 miles)

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow Gr$ (470 miles)

Both end in Gr, but the second is redundant because it is longer

Forgetting the Past

If a search algorithm does not remember where it has already been, it may

- fall into a cycle
- Explore redundant paths

This wastes time and memory and may prevent finding the optimal solution

Scanned with CamScanner

3 Approaches to handle Redundant Path / cycle / forgetting the past

1. Remember all visited states

Stategy: Maintain a visited set or list to avoid revisiting states

Used in: A*, Uniform Cost Search, Best-First Search

Benefit: Prevents cycles and redundant paths, ensures optimality and completeness

Trade-off: Requires more memory to store visited states

2. Ignore the Past

stategy: Simply keep visiting the nodes without checking

Used in: Tree Search algorithms

Benefit: Simpler implementation, lower memory usage

Trade-off: Can revisit the same state multiple times, leading to infinite loops

3. Selective Memory or Heuristic Pruning

Stategy: Use heuristics or domain-specific rules to prune paths without full state tracking

Used in: Hill climbing, Beam Search

Benefit: Fast and memory-efficient in large spaces

Trade off: May miss optimal solutions or get stuck in local minima.

Tree Search	Graph Search
I. Does not check for repeated states	I. Check for repeated states using an visited list
II. Uses less memory	II. Uses more memory
III. May create cycles and redendant paths	III. Avoid cycles and redendant paths
IV. Not always complete	IV. If a solution exists, it will find one
V. Not guaranteed optimal	V. Can be optimal
VI. Used in DFS, BFS	VI. Used in UCS, A*, Best-First Search

Scanned with CamScanner

Time and Space Complexity in Search

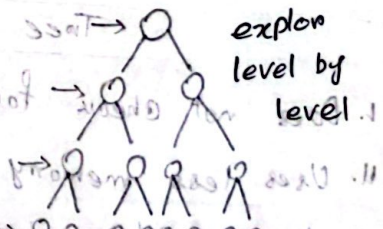
1. Explicit state space (Graph is given)
   - Complexity is measured by the size of the graph
   - $|V| + |E|$
     - $|E|$ — number of edges (state/action pairs)
     - $|V|$ — number of vertices (state)

2. Implicit state space (Graph not given, generated by actions) AI Search problem
   - Complexity is measured in terms of three key parameters
   - b (Branching factor): Average number of successors of a node
   - d (Depth of optimal solution): The number of steps in the shortest solution path
   - m (Maximum depth): The maximum length of any path in the state space.

Uninformed search also known as blind search is a type of search algorithm that explores a problem space without any domain-specific knowledge or heuristics

Type of uninformed search
- Breadth-first search (BFS) → complete, optimal (unit cost), but high memory — queue
- Uniform-cost search (UCS) → complete, optimal (any cost) Dijkstra, priority queue
- Depth-first search (DFS) → low memory, but not complete/optimal — stack
- Depth-limited search (DLS) → DFS with cutoff depth
- Iterative deepening search (IDS) → complete, optimal, low memory

Breadth First Search → Unweighted



Working
- Starts at the root node (initial state)
- Expands all nodes at depth 0, then all at depth 1, then depth 2 and so on
- Visits node level by level
- Uses a FIFO queue to ensure nodes are visited in the order they were generated.

Scanned with CamScanner

Properties

Completeness : Yes, if branching factor b is finite. Even in infinite state spaces

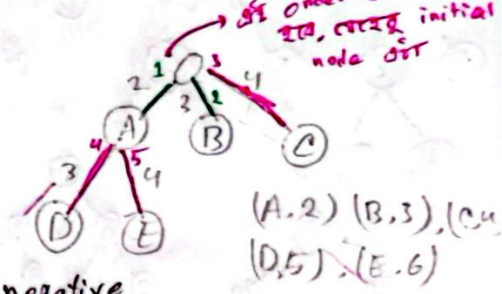
Optimality : Yes, if all step costs are equal (cost = 1)

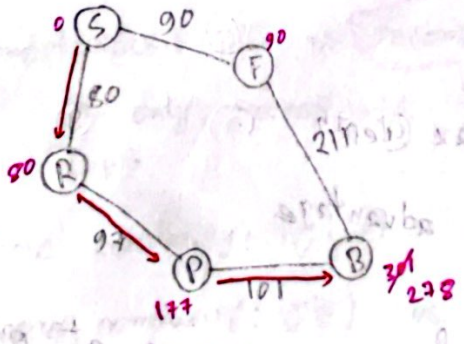
Time complexity : $O(b^d)$. b - Branching factor d - depth of solution

Space " : $O(b^d)$. due to storing all nodes at each level

Uniform Cost Search (UCS) / Dijkstra → weighted

- Finds the least-cost path from a start node to a goal node in a weighted graph
- Variant of Dijkstra algorithm
- Guarantees optimal solution if step costs are non-negative
- Explores nodes with the lowest cumulative cost first
- Uses a priority queue to manage nodes





	S	R	F	P	B
S	0	80	90	-	-
R	0	80	90	177	-
F	0	80	90	177	301
P	0	80	90	177	278

minimum cost path S → R → P → B = 0 + 80 + 97 + 101 = 278

Properties

Completeness : Yes, (if all step costs $\geq \varepsilon > 0$). and never get stuck on an infinite path

Optimality : Yes, always finds the least-cost path to the goal

Time complexity : $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$ C^* - cost of optimal solution ε - minimum step cost

Space complexity : Can be high due to storing all frontier nodes

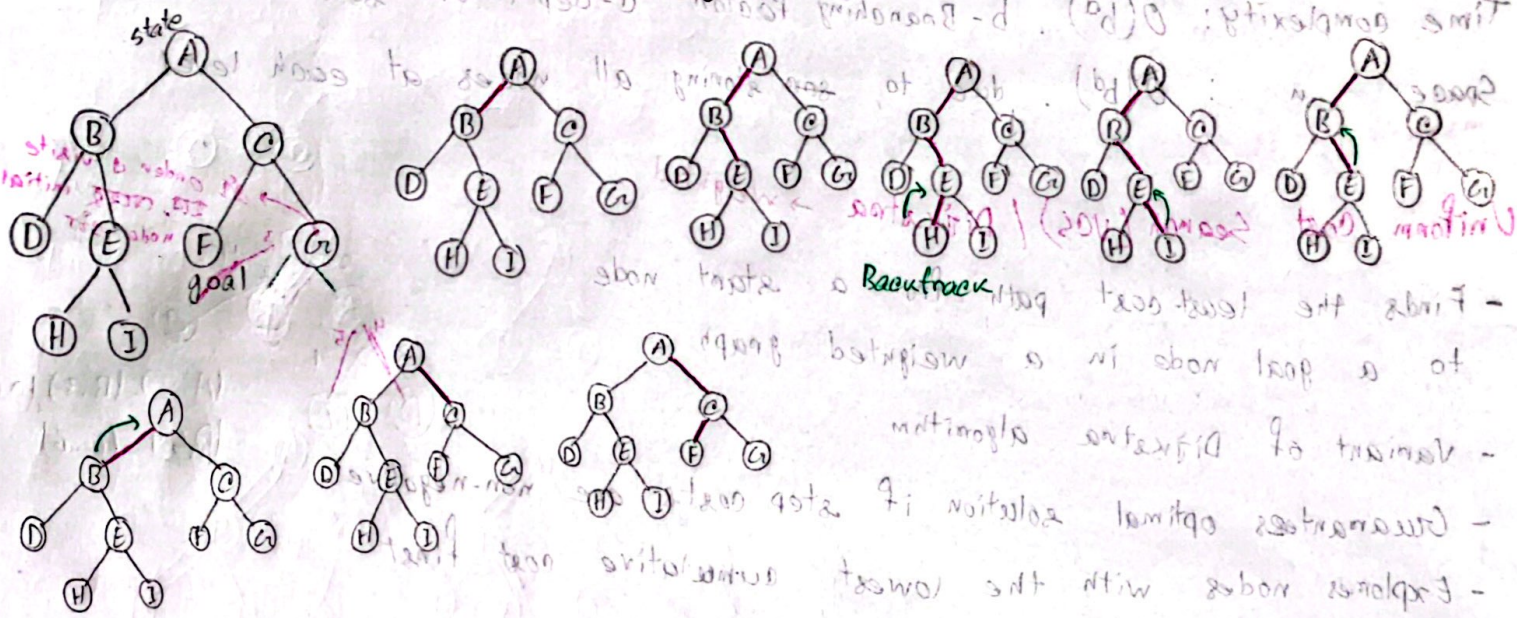
Queue Type : Priority Queue, ordered by cumulative cost $g(n)$

Scanned with CamScanner

Scanned with CamScanner

Depth-First Search

- Explores the deepest ~~unexp~~ unvisited node first
- Use a LIFO stack to manage unvisited node
- Backtracks when a path is fully visited or hits a dead end



Properties

Completeness : Yes, for finite tree

Optimality : No, returns the first solution it finds

Time complexity : $O(b^m)$, b- Branching factor, m-is max depth

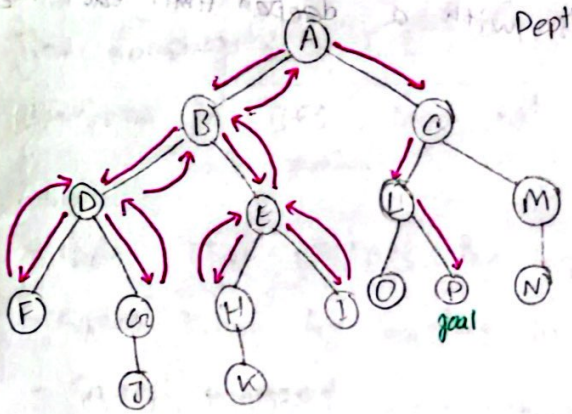
Space " : $O(bm)$, which is linear, a major advantage

Backtracking

- Generates one successor at a time instead of all successors at once
- Each partially expanded node keeps track of which successor to generate next
- Successors are generated by modifying the current state directly, instead of creating brand-new copies of states.
- Space complexity $O(m)$. Stores only one state and current path.

Depth Limited Search (DLS)

~ is a variant of DFS. that imposes a limit on the depth of the search tree. This limit helps prevent the search from going too deep and potentially getting stuck in an infinite loop. It is particularly useful in problems where the search space is vast or infinite.



Depth limit = 3

- infinite deepth এ যাবে না
- Depth limit এর মাঝে Goal থাকলে খুঁজে পাবে, এর বাইরে থাকলে পাবে না
- Depth limit এর নিচে যেতে পারে না

Depth Limitation: The primary characteristic of DLS is the depth limit (L). The algorithm explores the search tree only up to a specified depth.

Completeness: DLS is incomplete if the solution exists beyond the depth limit. It only guarantees finding a solution if it exists within the specified depth.

Time complexity: $O(b^L)$, b- Branching factor, L- Depth Limit.

Space complexity: $O(L)$. as it only needs to store the nodes in the current path.

Optimality: Not optimal.

Application: Use where search space is too large, such as infinite or very deep search space

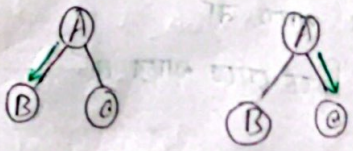
Scanned with CamScanner

Iterative Deepening Search (IDS)

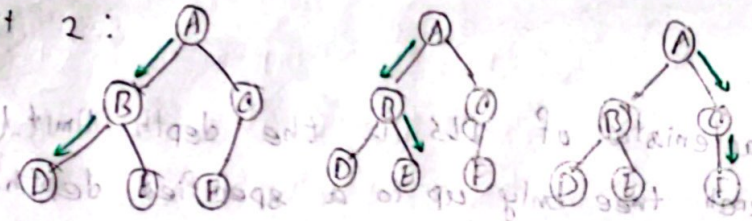
- Performs depth-limited search repeatedly, increasing the depth limit incrementally
- First with limit L=0, then L=1, L=2, until the goal is found
- Each iteration is a complete DFS up to the current depth
- Reuses the same DFS logic but restarts with a deeper limit each time.

Limit 0 : (A)

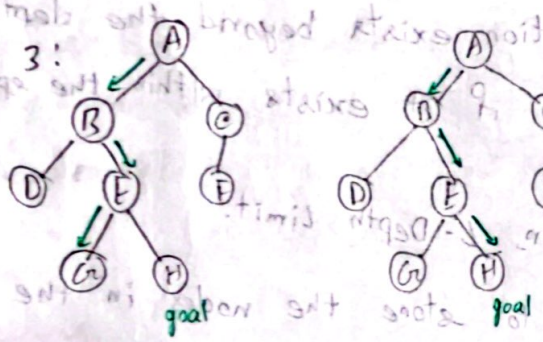
Limit 1 :



Limit 2 :



Limit 3 :



Properties

Complete : Yes. always reach the goal

Optimal : Yes

Time complexity : $O(b^d)$ if solution exists at depth d

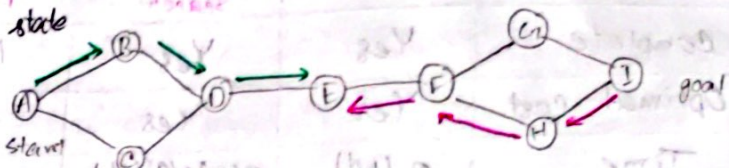
$O(b^m)$ if no u . u

Space u : $O(b \cdot d)$. linear like DFS

Scanned with CamScanner

Bidirectional Search

- A search strategy that runs two simultaneous searches
  1. Forward search : From the start state to goal state
  2. Backward search : From goal state to start state
- Stops when two search meet



Time complexity : $2\ b^{d/2}$

- Complete in BFS, May not complete if use DFS

Advantages

- Faster than BFS/DFS for large problem
- Saves time by meeting in the middle
- Save memory
- Complete, if BFS is used
- Optimal, if BFS is used and step costs are uniform

Dis

- Works only if the goal state is known in advance
- Hard to implement
- can get stuck in infinite loops
- Backward search is tricky

When to Use

- When start and goal states are unique and clearly defined
- when branching factor is the same forward and backward
- Good for problems like.
  - Route finding (maps, graph)
  - Sliding block puzzles
  - Word ladder puzzles

Avoiding Repeated States

- Must check if a state was already visited
- Failure to check can turn linear problems into exponential ones
- Easier when actions are reversible.

Scanned with CamScanner

	BFS	Uniform Cost Search	DFS	Depth Limited Search	Iterative Deepening Search	Bidirectional Search
Complete	Yes	Yes	No	No	Yes	Yes
Optimal Cost	Yes	Yes	No	No	Yes	Yes
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$

Informed Search also called heuristic search is a type of search algorithm that uses domain-specific knowledge in form of a heuristic function $h(n)$, to guide the search process toward a goal more efficiently than uninformed (blind) search.

Heuristic function $h(n)$: estimated cost of the cheapest cheapest path from the current node n to a goal state

- not always optimal but provides a good guess

properties

- faster than uninformed search
- memory-efficient compared to uninformed search
- Quality depends on how good the heuristic is
  - A good heuristic → closer to the real cost
  - A bad " → can mislead the search

Type of informed search

- Greedy Best-First search → $f(n) = h(n)$. Uses only heuristic value. Fast but not optimal
- A* search → $f(n) = g(n) + h(n)$ → Uses both path cost and heuristic. Complete and optimal
- Iterative Deepening A* (IDA*) → memory-efficient version of A*
- Memory-Bounded A* (SMA*) → Handles large state spaces where memory is limited
- Beam search → A heuristic search that expands only the best w nodes at each level

Scanned with CamScanner

(Greedy)

Best First Search (Informed, heuristic)

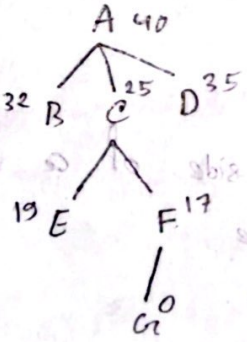
Uses only the heuristic function $f(n) = h(n)$

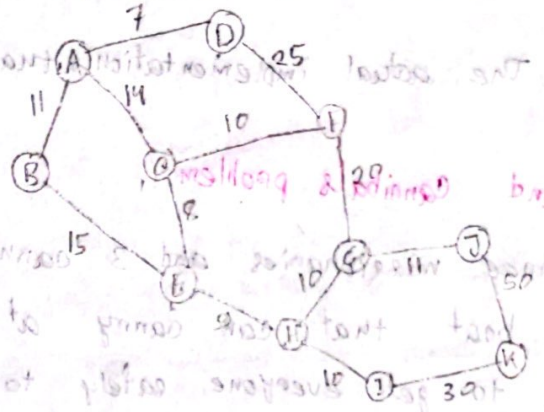
Property

Completeness : Yes, in finite state spaces
NO, in infinite loop

Optimality : Not optimal

Time : $O(|V|)$ → V number of states.





Straight line distance

A → G = 40
B → G = 32
C → G = 25
D → G = 35
E → G = 19
F → G = 17
H → G = 10
G → G = 0

∴ Path : A → C → F → G → not optimal
Optimal path : A → C → E → H → G

Greedy ≠ Optimal

adv

- Combines the best ideas of BFS and DFS
- More efficient than BFS and DFS in many cases
- Works well with good heuristics

Dis

- Can behave like an unguided DFS in the worst case
- May get stuck in a loop
- Not optimal

Scanned with CamScanner

F

(1b) An Intelligent agent is a program or system that can think about what's happening around it and take actions to reach its goal.

Components of an Intelligent Agent

I. Sensors: The agent uses input mechanisms to observe its environment.

II. Reasoning: The agent processes the information by its logic

~~III. Decision-making~~

III. Agent Function: The brain that maps perception to actions

IV. Agent Program: The actual implementation that runs on the machine

(2a) Missionaries and Cannibals problem

- You have three missionaries and 3 cannibals on one side of a river
- There's one boat that can carry at most 2 people
- The goal is to get everyone safely to the other side

Constraints

- If cannibals ~~outnumbe~~ more than missionaries on either bank, the missionaries get eaten
- The boat can't cross empty. It must carry at least one person
- we must find a sequence of crossing that avoids any unsafe situations.

state $\frac{mmm}{kkk}\,||$ → $\frac{mmm}{k}\,||\,kk$ → $\frac{mmm}{kk}\,||\,k$ → $\frac{mmm}{}\,||\,kkk$ → $\frac{mmm}{k}\,||\,kk$

$\frac{m}{k}\,||\,\frac{mm}{kk}$ → $\frac{mm}{kk}\,||\,\frac{m}{k}$ → $kk\,||\,\frac{mmm}{k}$ → $kkk\,||\,\frac{mmm}{}$ → $k\,||\,\frac{mmm}{kk}$ → $kk\,||\,\frac{mmm}{k}$

$||\,\frac{mmm}{kkk}$

goal

Scanned with CamScanner

Water Jug Problem

We have 2 jugs with different capacities and an unlimited water supply. The task is to measure out a specific amount of water by filling, emptying or pouring water between the jugs. There are no measurement marking on the jugs.

Allowed Operations

- Fill a jug completely
- Empty a jug
- Pour water from one jug into the other until one is full or the other is empty.

* You are provided with 2 jugs : one with capacity of 4 gallons and another with capacity of 3 galons
  - You have access to an unlimited supply of water via a pump
  - It can pour water onto the ground to empty a jug
  - The jugs do not have any measuring markings.

a. How will you measure exactly 2 gallons of water in the 4 gallon jug

We can measure exactly 2 gallons in 4-gallon jug by using a sequence of operations involving filling, pouring and emptying the 2 jugs. The key is to transfer water between the jugs strategically until the desired amount is reached.

b.

Scanned with CamScanner

b. Step by Step Solution

I. Fill 4 gallons to jug A : (0,0) → (4,0)

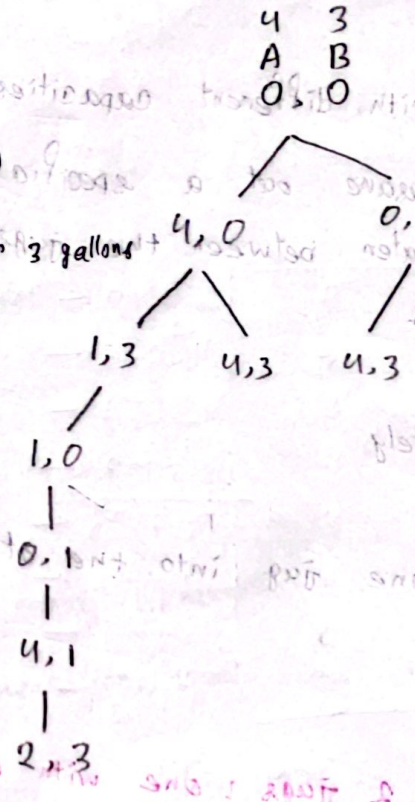
II. Pour A → B : (4,0) → (1, 3) transfer 3 gallons

III. Empty jug B : (1,3) → (1,0)

IV. Pour A → B : (1,0) → (0,1) transfer 1

V. Fill 4 gallons to jug A : (0,1) → (4,1)

VI. Pour A → B : (4,1) → 2, 3



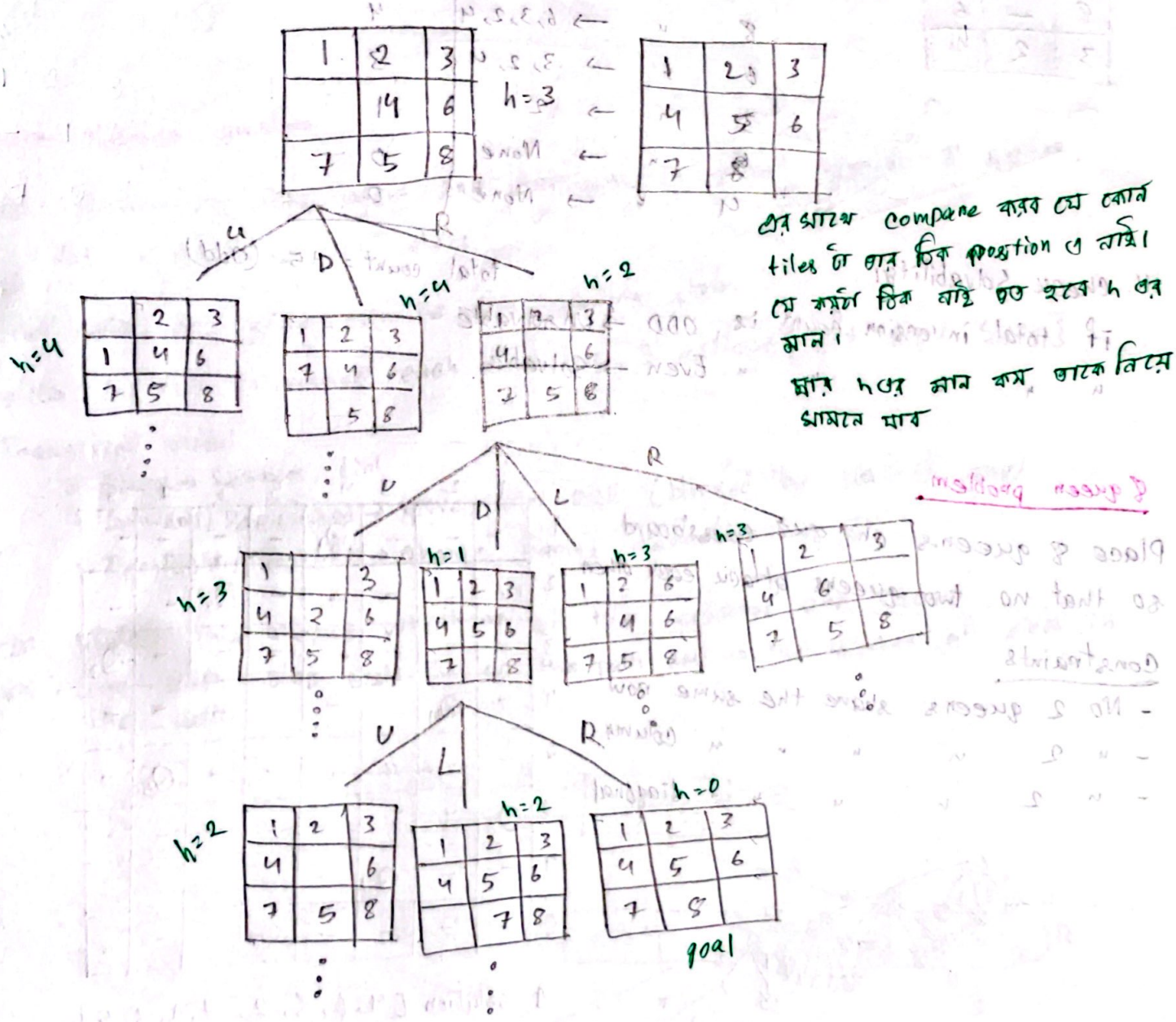
C. Production rules that define valid actions

1	(x,y) if x<4	(4,y)	Fill the 4 gallon jug
2	(x,y) if y<3	(x,3)	" " 3 " "
3	(x,y) if x>0	(x-d, y)	Pour some water out of the 4 gallon jug
4	(x,y) if y>0	(x, y-d)	" " " " " " 3 " "
5	(x,y) if y>0	(0, y)	Empty 4 gallon jug
6	(x,y) if x>0	(x,0)	" 3 " "
7	(x,y) if x+y>=4 and y>0	(4, y-(4-x))	Pour water from the 3 gallon jug into the 4 gallon jug until the 4 gallon jug is full
8	(x,y) if x+y>=3 and x>0	(x-(3-y), 3)	Pour water from the 4 gallon jug into the 3-gallon jug until the 3 gallon jug is full
9	(x,y) if x+y<=4 and y>0	(x+y, 0)	Pour all the water from the 3 gallon jug into the 4 gallon jug
10	(x,y) if x+y<=3 and x>0	(0, x+y)	Pour all the water from the 4 gallon jug into the 3 gallon jug
11	(0,2)	(2,0)	Pour the 2 gallons from 3 gallon jug into the 4 gallon jug
12	(2,y)	(0,y)	Empty the 2 gallons in the 4 gallon jug on the ground.

Scanned with CamScanner

8 Puzzle Problem

- It consists of a 3×3 board with 8 numbered tiles (1-8) and one empty space
- The tiles can slide up, down, left, right into the empty space
- The initial state is any random arrangment of tiles
- The goal state is usually arranged like this $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & - \end{bmatrix}$



এর সাথে compare করব যে কোন tiles টা তার ঠিক position এ নাই। যে সংখ্যা ঠিক নাই তত হবে h এর মান।

যার h এর মান কম তাকে নিয়ে সামনে যাব

Rules for 8 Puzzle Solvability / Unsolvability

I. Flatten the Puzzle! Read the tiles from left to right, row by row to form a 1D list.

II. Count Inversions:

1	5	7
8	—	6
3	2	4

			inversion count
1	Precedes →	None	0
5	" →	3, 2, 4	3
7	" →	6, 3, 2, 4	4
8	" →	6, 3, 2, 4	4
6	" →	3, 2, 4	3
3	" →	2	1
2	" →	None	0
4	" →	None	0

total count = 15 (Odd)

III. Check Solvability:

if total inversion count is ODD → unsolvable

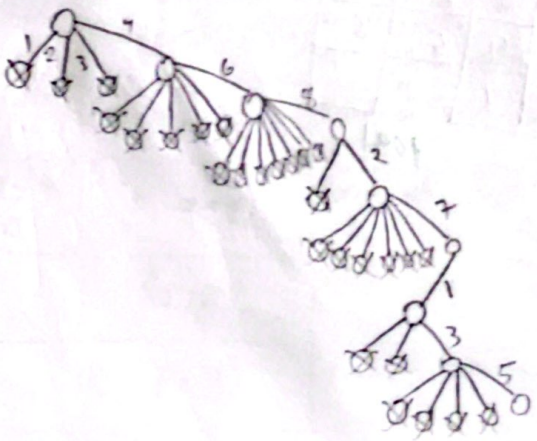
" " " " " Even → Solvable

8 queen problem

Place 8 queens on 8×8 chessboard so that no two queens attack each other

Constraints

- No 2 queens share the same row
- " 2 " " " " column
- " 2 " " " " diagonal



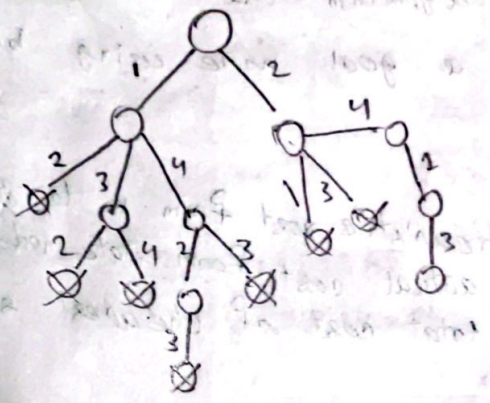
	1	2	3	4	5	6	7	8
1	-	-	-	Q_1	-	-	-	-
2	-	-	-	-	-	Q_2	-	-
3	-	-	-	-	-	-	-	Q_3
4	-	Q_4	-	-	-	-	-	-
5	-	-	-	-	-	-	Q_5	-
6	Q_6	-	-	-	-	-	-	-
7	-	-	Q_7	-	-	-	-	-
8	-	-	-	-	Q_8	-	-	-

1 solution [4, 6, 8, 2, 7, 1, 3, 5]

2 " [5, 3, 1, 7, 2, 8, 6, 4]

Scanned with CamScanner

4 queen problem



	1	2	3	4
1	-	Q_1	-	-
2	-	-	-	Q_2
3	Q_3	-	-	-
4	-	-	Q_4	-

Vacuum cleaner problem

I. State: Show where the agent and dirt are. In 2-cell world → 8 states.
For n cells → $n \times 2^n$ states

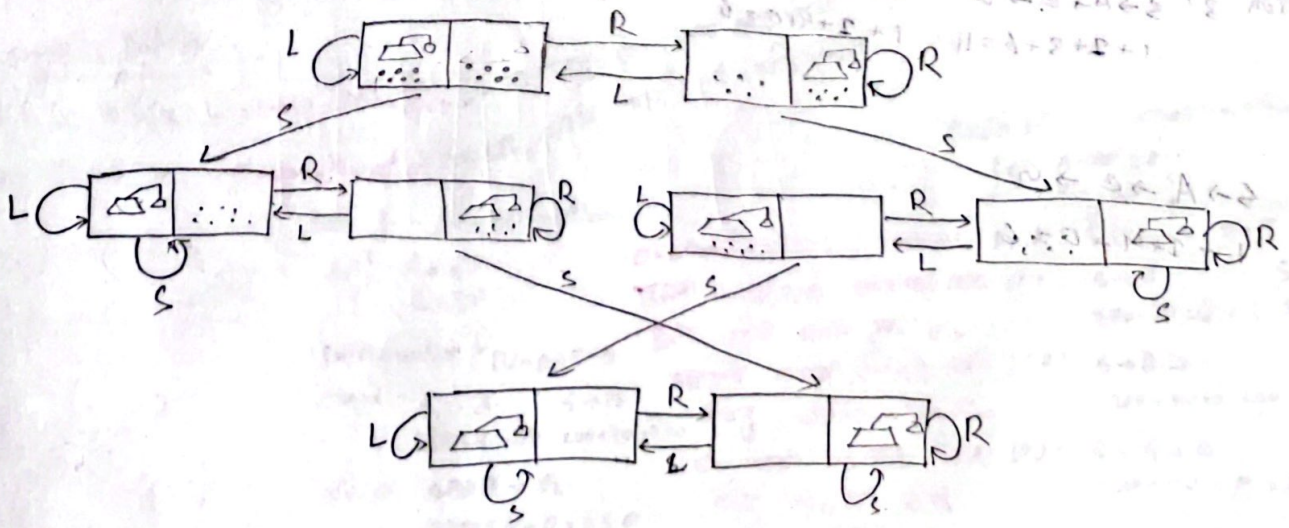
II. Initial state: Any state can be the starting state

III. Action: In this environment each state has 3 actions {Left, Right, Suck}

IV. Transition model
- Suck → Removes dirt
- Forward / Backward → move one cell (blocked by wall if any)
- TurnRight / TurnLeft → Change facing direction by 90°

V. Goal State: This checks whether all the squares are clean

VI. Path cost: each step costs 1. so the path cost is the number of steps in the path.

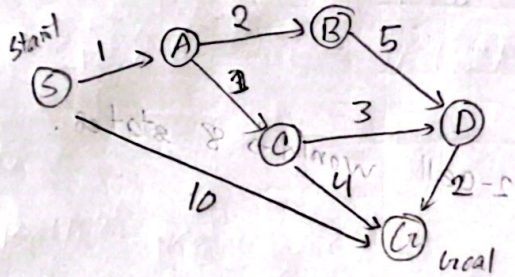


A* Search → guarantee করে এর খরচ optimal solution এসেছে

is a best-first search algorithm that finds the shortest path from a start node to a goal node using both actual cost and heuristic cost.

$f(n) = g(n) + h(n)$ ← fitness number

- $h(n)$ → heuristic cost from n to goal node
- $g(n)$ → actual cost from start node to n
- $f(n)$ → total cost of cheapest solution



n	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

Initialization: $f(S) = g(S) + h(S)$
$= 0 + 5$
$= 5$

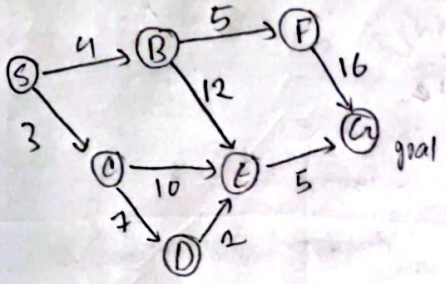
Iteration 1: $S \to A$: $1 + 3 = 4$; $S \to G$: $10 + 0 = 10$

Iteration 2: $S \to A \to B$: $1 + 2 + 4 = 7$; $S \to A \to C$: $1 + 1 + 2 = 4$; $S \to G$: 10

Iteration 3: $S \to A \to C \to D$: $1 + 2 + 3 + 6 = 11$; $S \to A \to C \to G$: $1 + 2 + 4 + 0 = 6$; $S \to A \to B$: 7 ; $S \to G$: 10

∴ $S \to A \to C \to G$
$1 + 1 + 4 + 0 = 6$

Scanned with CamScanner



h	
S	14
B	12
C	11
D	6
E	4
F	11
G	0

Initialization : $f(S) = g(S) + h(S)$
$= 0 + 14 = 14$

Iteration 1: $S \to B$: $4 + 12 = 16$; $S \to C$: $3 + 11 = 14$

Iteration 2: $S \to C \to E$: ~~14+10+4=~~ $3+10+4=17$; $S \to C \to D$: ~~14+7+6=~~ $3+7+6=16$; $S \to B$: 16

Iteration 3 : $S \to C \to D \to E$: $3+7+2+4=16$; $S \to B \to F$: $4+5+11=20$; $S \to B \to E$: $4+12+4=20$; $S \to C \to E$: 17

Iteration 4 : $S \to C \to D \to E \to G$: $3+7+2+5+0=17$; $S \to C \to E \to G$: $3+10+5+0=18$

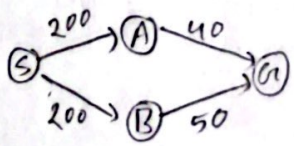
∴ Shortest path : $S \to C \to D \to E \to G$

Admissible মানে সব সময় আমাকে optimal solution দিবে।

A* Admissibility

$h(n) \leq h^*(n)$ - underestimation → যা দাম, তার থেকে কম ধরা

$h(n) \geq h^*(n)$ - Overestimation → যা দাম, তার থেকে বেশি ধরা



n	h(n)
S	5
A	– (নেওয়া যাবে না)
B	–
G	0

case I : Overestimation

Let, A = 80
B = 70

Initialization $f(S) = 0+5 = 5$

Ite-1 : $S \to A$: $200+80=180$; $S \to B$: $200+70=270$

Iter 2 : $S \to B \to G$: $200+50+0=250$

$S \to B \to G$

case II : Underestimation

Let, A = ~~80~~ 30
B = 20

overestimation এ সমস্যা হলো optimal path আমার জন্য It 1 : $S \to A$: $200+30=230$; $S \to B$: $200+20=220$

আর চেক করে না। তখন বারবার কমটি তোমে যাবে, কিন্তু underestimation এ সমস্যা হলো বার বার আমার কম পাওয়ার জন্য চেক করতে থাকে

It 2 : $S \to B \to G$: $200+50+0=250$; $S \to A$: 230

It 3 : $S \to A \to G$: $200+40+0=240$

∴ $S \to A \to G$

Property of A*

- A* is complete when branching factor is finite.
- It give optimal soulition when is achived
  - Admissible heuristic $h(n) \leq h^*(n)$
  - Consistency $h(n) \leq c(n, a, n') + h(n')$
- Time complexity $O(b^d)$
- Space " $O(b^d)$

Adv

- Better performance than most other search algorithm
- always give optimat solution
- Guaranteed the completeness
- Suitable for solving complex real-world problems
- widely used in games, robot navigation, and web-based maps to find the shortest path

Dis

- If overestimation happen, it may not give optimal soulation
- More complex
- requires large memory
- Efficiency depends on the quality of heuristic function
- May return the first found path, ignoring other optimal one.

weighted A* $f(n) = G(n) + W \times h(n)$ where $W > 1$

A* Search:	$g(n) + h(n)$	$W = 1$
Uniform-cost Search:	$g(n)$	$W = 0$
Greedy best-first Search:	$h(n)$	$W = \infty$
Weighted A* Search:	$g(n) + W \times h(n)$	$1 < W < \infty$

Type of sub-optimal Search algorithm

1. Bounded Sup-optimal Search: Finds a solution within a factor W of the optimal cost → weighted A*.

2. Bounded-Cost Search: Ensures solution cost ≤ a fixed bound c
→ Iterative Deepening Search (IDS) with cost bound

3. Unbounded cost ~ : Ignores cost, focuses only on speed → Greedy Best-first se

Memory-Bounded Search is a type of search algorithm designed to operate efficiently under limited memory conditions.

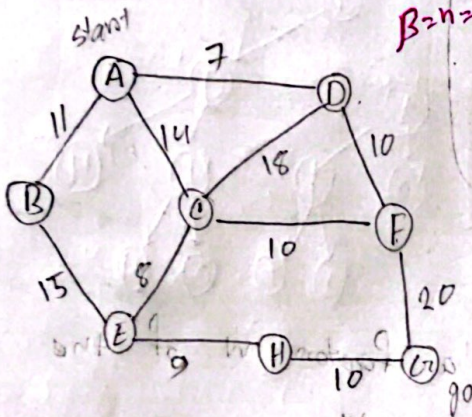
- aim to balance optimality, completeness, and memory efficiency
- restrict memory usage to a fixed limit
- Iterative Deepening A*, Simplified Memory-Bounded A*

Scanned with CamScanner

Beam Search Algorithm → এটা একটা Best First Search, শুধু এখানে queue তে শুধু B এর সমান value রাখব

- Take care of space complexity
- Beam width (B)

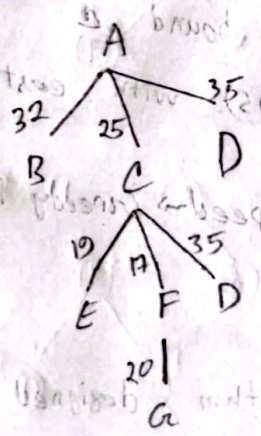
$B = n = 2$

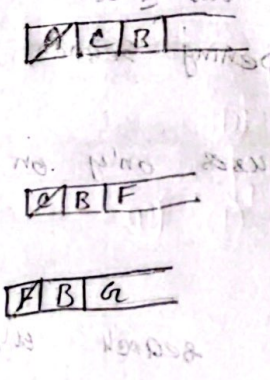


n	h(n)
A	40
B	32
C	25
D	35
E	19
F	17
H	10
G	0

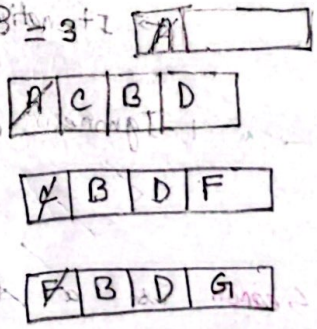
Best First Search	Beam Search
Time complexity $O(b^t)$	constant
Space = $O(b^t)$	=
complete =	Incomplete

Beam Search এর এর থেকে বড় সমস্যা হলো যদি Dead end এ চলে গেলে আর gole এ যেতে পারে না





$B = 3+2$



B এর value

infinite = Best first search

finite = Beam search

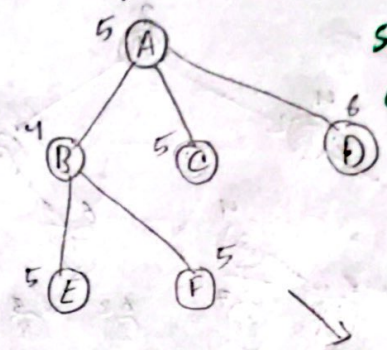
B = 1 হলে Hill climbing algo

→ local search algo greedy approach, no backtrack

Hill Climbing Algorithm is a heuristic search algorithm that continuously moves toward the neighboring state with a higher value, hoping to reach the peak or optimal solution

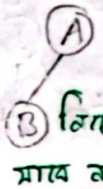
- Local Search Algorithm
- Greedy Approach
- Beam value (β) = 1
- No backtracking

Main adv → less memory need



Success = 14%
Get stuck = 86%
but need less memory

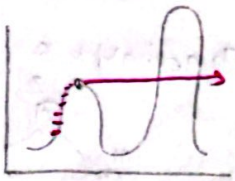
Hill climbing এ খুব সময় নিচের থেকে better ans খুঁজতে থাকে, যদি আর কাছে থাকে, আর না পেলে ঐ লেভেলে ans হিসাবে নিবে।



নিচে আর যাবে না, একেই Ans ধরে নিবে

Problems

1. Stuck in local Maximum

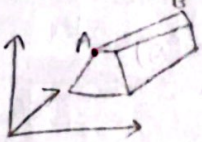


খুঁজতে খুঁজতে আর কাছে যখন আর better ans পাবে না, তখনই থেমে যাবে। কিন্তু আর থাকে কাছে আর ভালো solution নিবে।

2. Plateau / Flat Maximum

এখানে same value থাকে তাই আর সামনে যায় না

3. Rige



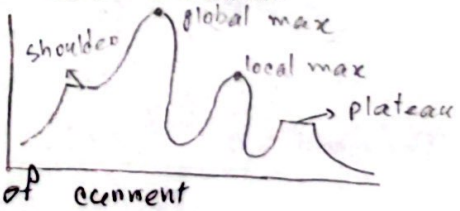
এখানে angle change হচ্ছে তাইও আর value update করবে না। এখানেই ans ধরে নিবে।

MID

Algorithm

function Hill_Climbing (Problem) returns a state that is a local maximum

```
current ← problem.Initial
while true
    neighbor ← a highest valued successor state of current
    if value(neighbor) ≤ value(current) return current
    current ← neighbor
```



Local Maximum

A point that is higher then all nearby states, but lower than the global maximum
- The algo may stuck here

Global Maximum

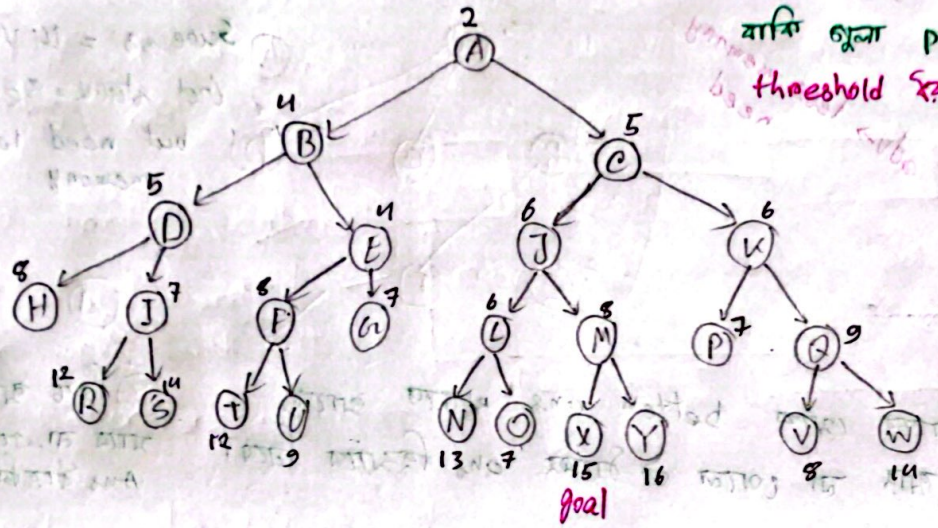
The highest possible point in the entire search space.

A Plateau is a flat area in the state-space, where it's neighboring states have the same value. Here algo can't decide which direction to move.

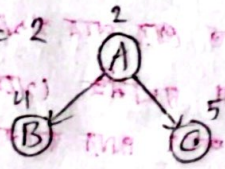
Scanned with CamScanner

Iterative Deepening A* Search

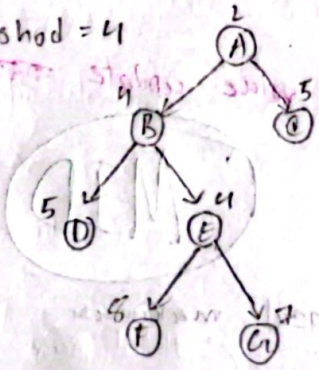
আমরা threshold এর মধ্যে বা তার চেয়ে কম value কে explore করব।
বাকি গুলা prune করব।
threshold হবে সবচেয়ে কম node এর value



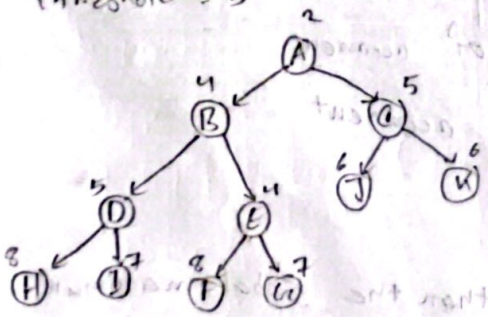
Threshold = 2



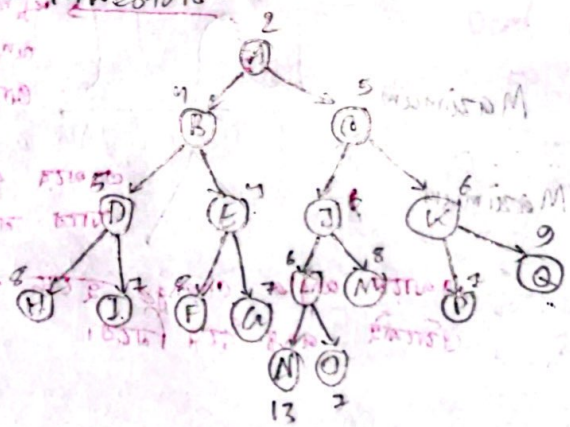
Threshold = 4



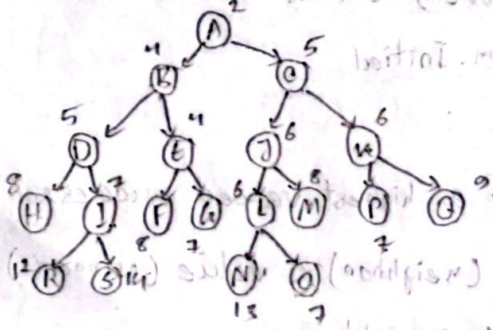
Threshold = 5



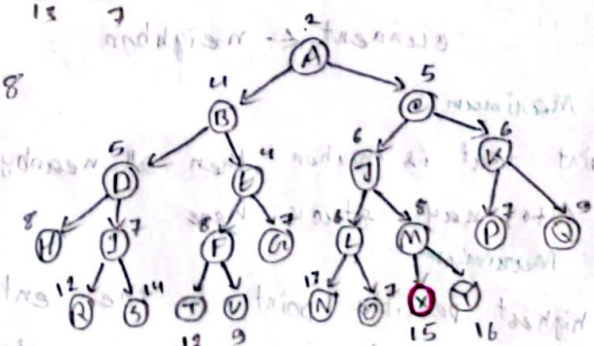
Threshold = 6



Threshold = 7



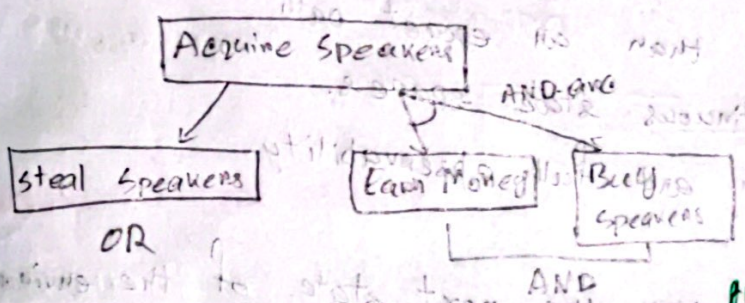
Threshold = 8

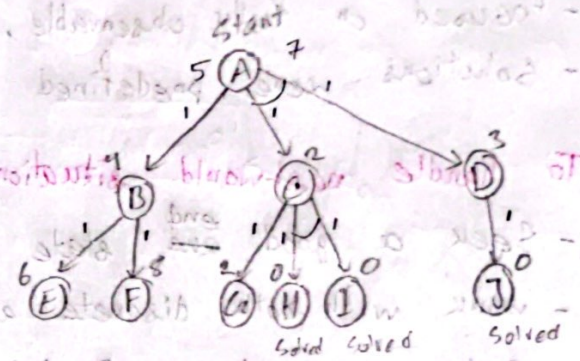


Scanned with CamScanner

AO* Algorithm

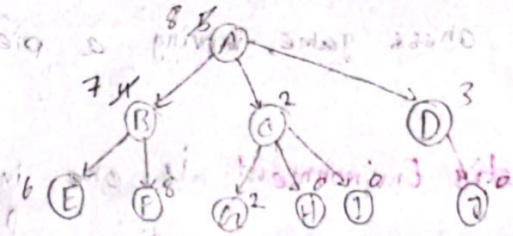
- Best First Search algorithm
- Uses the concept of AND-OR graphs
- AND-OR graphs → Specialized graphs (decomposition)
- AND-set of task, OR-Single task





- Knowledge-based search technique
- Best path → heuristics
- Goal optimal solution → Stop Search

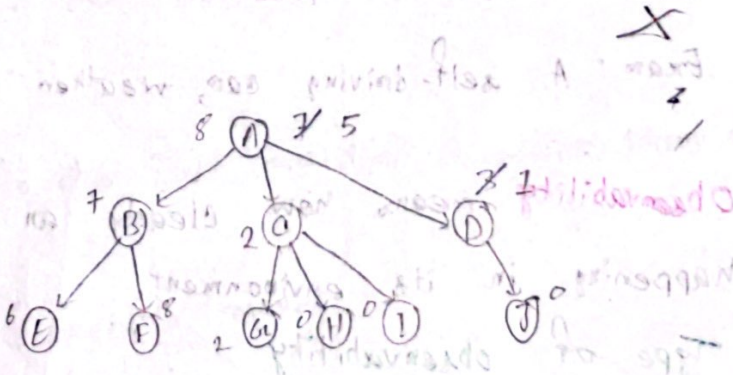
h(n) দেয়া না থাকলে 1 ধরে নিব।



Step 1: A → B : 1 + 4 = 5
A → C → D : 1 + 2 + 1 + 3 = 7

Step 2: B → E : 1 + 6 = (7)
B → F : 1 + 8 = 9

এর পরে আর কিছু নাই, তাই এটাই goal state
এখন 7 দিয়ে update করতে হবে।
A → B
1 + 7 = 8 ← heuristic value 7 হবে



Step 3: C → G : 1 + 2 = 3
C → H → I : 1 + 0 + 1 + 0 = (2)

2 দিয়ে update করব

D → J
1 + 0 = (1)

3 কে 2 দিয়ে update করলে
A → C → D
1 + 2 + 1 + 0 = 5

path A → C → D = 5

From Simple to Realistic Environments in AI

- Focused on fully observable, deterministic and known environments
- Solutions were predefined sequences of actions

To handle real-world situations, AI must

- Seek a good end state rather than an exact path
- work with both discrete and continuous state spaces.
- Relax assumptions of determinism and full observability

Deterministic Environment is one in which the next state of the environment is completely determined by the current state and the agent's action, there is no randomness or uncertainty involved

Exam: In a chess game, moving a piece always produces the same result.

Non-deterministic Environment is one in which the results of actions are uncertain, the same action performed in the same state may lead to different possible outcome.

Exam: A self-driving car, weather prediction, Robot navigation

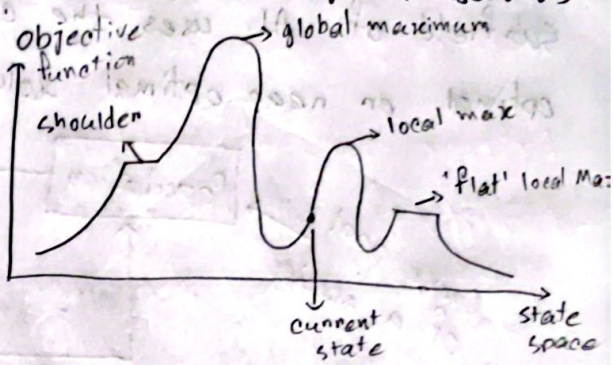
Observability means how clearly an agent can see or sense what's happening in its environment

Type of observability

- **Fully Observable Environment:**
  The agent has compete information about the environments state
  Ex: A chess game
- **Partially Observable Environment**
  The agent has limited or incomplete information about the environ-
  Ex: A self-driving car

Scanned with CamScanner

Local Search Algorithm is a search method that start from an initial state and move to neighboring states in order to find an optimal solution without keeping track of the entire search path



Main advantages

- Uses very little memory
- can find good solutions quickly

Variations of Hill climbing

I. Steepest-Ascent Hill climbing

- looks at all neighbors and moves to the one with the biggest improvement
- Fast but can easily get stuck in local maxima

Ex: In 8-Queen, checks all 56 moves and picks the best one

II. Stochastic Hill climbing

- Randomly chooses one of the better moves
- Slower but sometimes finds better results than steepest-ascent
- Helps avoid getting trapped in local maxima

III. First-choice Hill climbing

- Generates random moves and picks the first one that it better than the current state
- Useful when there are many possible moves
- Faster but may get stuck

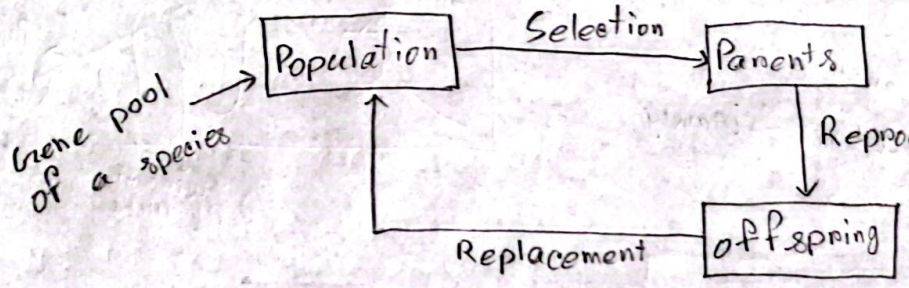
IV. Random-Restart Hill climbing

- Runs hill climbing many times from different random starting points
- If one try get stuck, it restarts
- Simple and effective way to find the global solution

Scanned with CamScanner

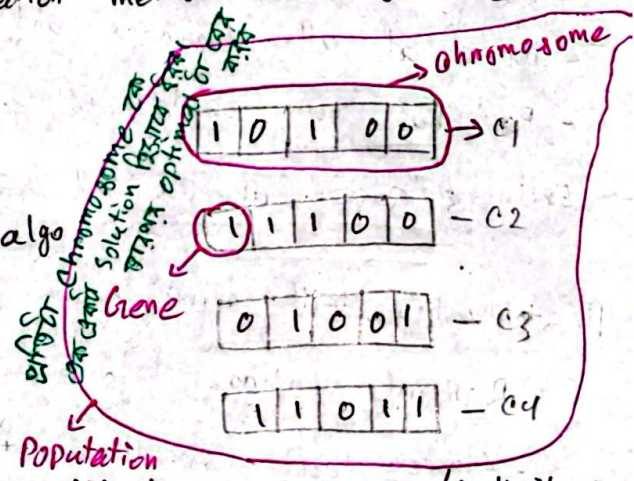
Evolutionary Computation

Subfield of AI uses the concept of biological evolution to find optimal or near optimal solutions for a given problem.



Steps of EA:
1. Initialization
2. Selection of chromosome
3. Crossover and mutation

Genetic Algorithm is a search and optimization method based on the principles of natural selection and genetics.

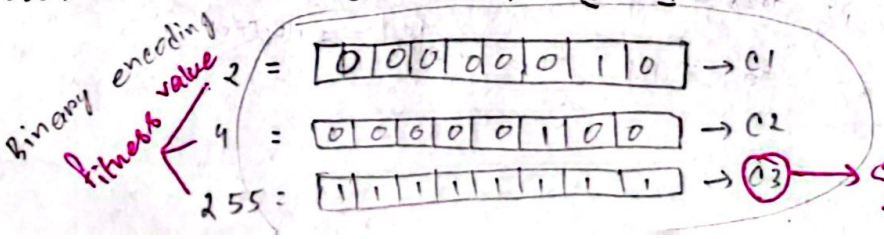


- First step of EA (GA)
- Six parameters are set before starting the algo
  1. No of genes per chromosome
  2. The code value
  3. The size of population per generation
  4. Crossover probability
  5. Mutation
  6. Termination Criteria and max no of generations.

bit stream = chromosome / individual
bit = Gene / allele
population = set of chromosome

There most widely used encoding of gene:
- Binary Encoding
- Real number Encoding
- Permutation encoding → travelling salesman

Example: maximize $f(x) = x^3$; $0 \le x \le 255$ ∴ $256 = 2^8$ → ∴ 8 bit



এটি হলো optimal, যেহেতু max কের বলতে হবে

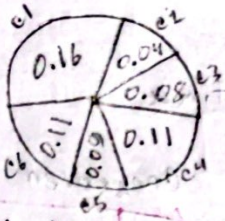
Q. Selection of chromosome

- Roulette wheel Selection
- Rank Selection
- Tournament "
- Random Selection

Roulette wheel Selection

Random selection of m chromosome from a big population (when fitness are not very different)

length of circumference ∝ fitness value



6-individual / chromosome

$c_1 - 6$
$c_2 - 5$
$c_3 - 5$
$c_4 - 4$
$c_5 - 4$
$c_6 - 5$

বেশি value ওয়ালা chromosome গুলোর ফিটনেস বেশি, সুতরাং selection এর chance ও বেশি

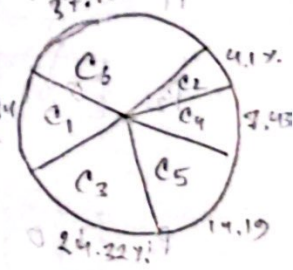
$c_1 - 3$
$c_2 - 0$
$c_3 - 1$
$c_4 - 1$
$c_5 - 2$
$c_6 - 2$

value কম হলে selection এর chance কমে যায়, bias করে না

* Determine percentage of Roulettle wheel and actual count for six chromosomes with fitness values of 19, 6, 36, 11, 21 and 55

যেই value তে সবচেয়ে 6 বার, সেটাই 6 টা chromosome আছে

Chromosome	Fitness, F	% of Roulette wheel	Probability P_i	Expected count ($P_i \times N_{ind}$)	Actual Count
c1	19	12.84%	0.1284	0.77	1
c2	6	4.1%	0.0405	0.243	0
c3	36	24.32%	0.2432	1.46	1
c4	11	7.43%	0.0743	0.44	1
c5	21	14.19%	0.1419	0.85	1
c6	55	37.16%	0.3716	2.23	2



$\sum F = 148$ $\quad P_i = \frac{F_i}{\sum F} \times 100\%$ $\quad P_i = \frac{F_i}{\sum F}$

Crossover : Generation of offspring/child

Mutation : Change characteristic of next generation

Extreme case of Crossover

Case I: If all chromosomes are selected for mating then all possible offspring are generated

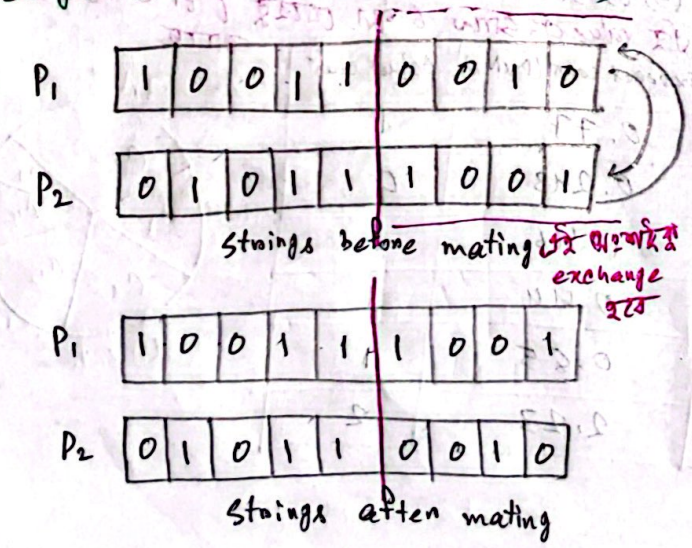
Case II: If no chromosomes is selected for mating, then all chromosomes of parents are copied for next generation.

Drawback of crossover: May stuck into local optima

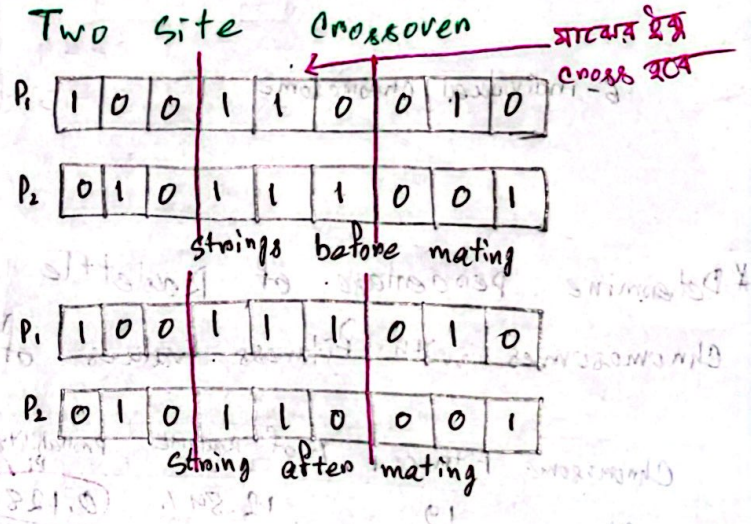
Type of Crossover

- Single-Site Crossover
- Two Site "
- Crossover Mask

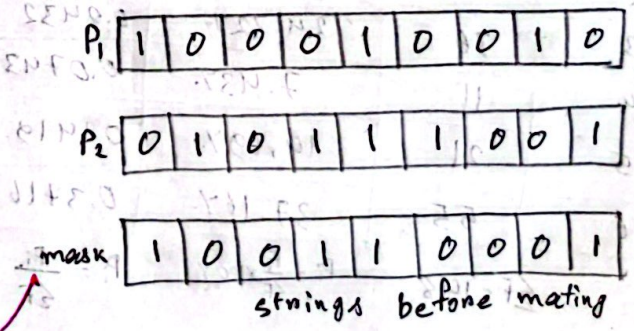
Single Site Crossover (at point 5)



Two Site Crossover



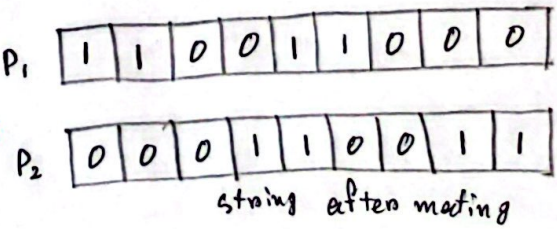
Crossover Mask



mask

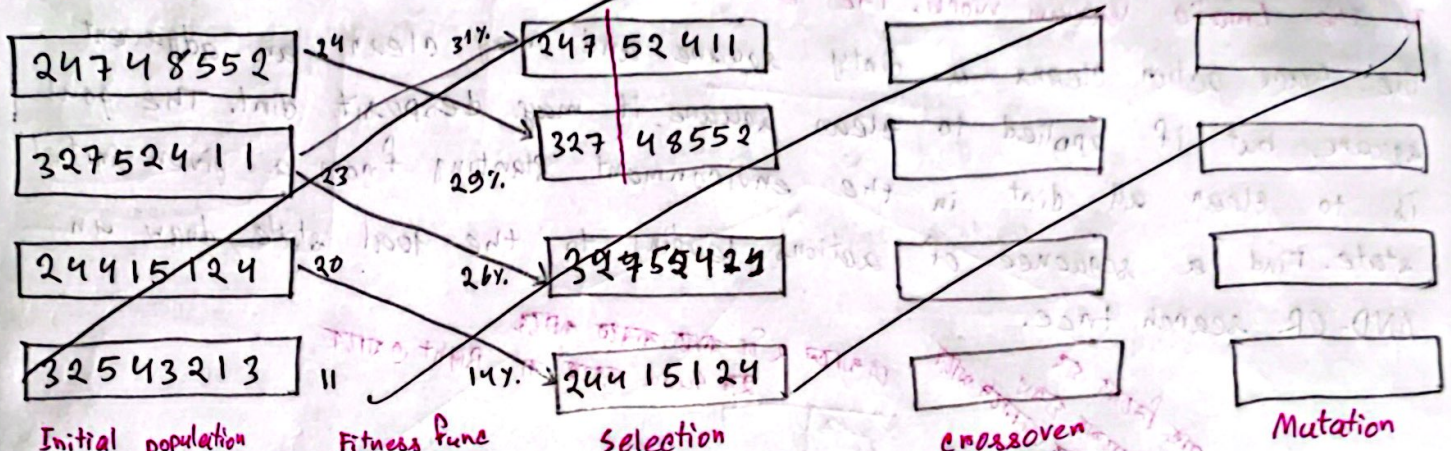
1 → যেখানে 1 আছে সেখানে নিজের জায়গা থাকবে

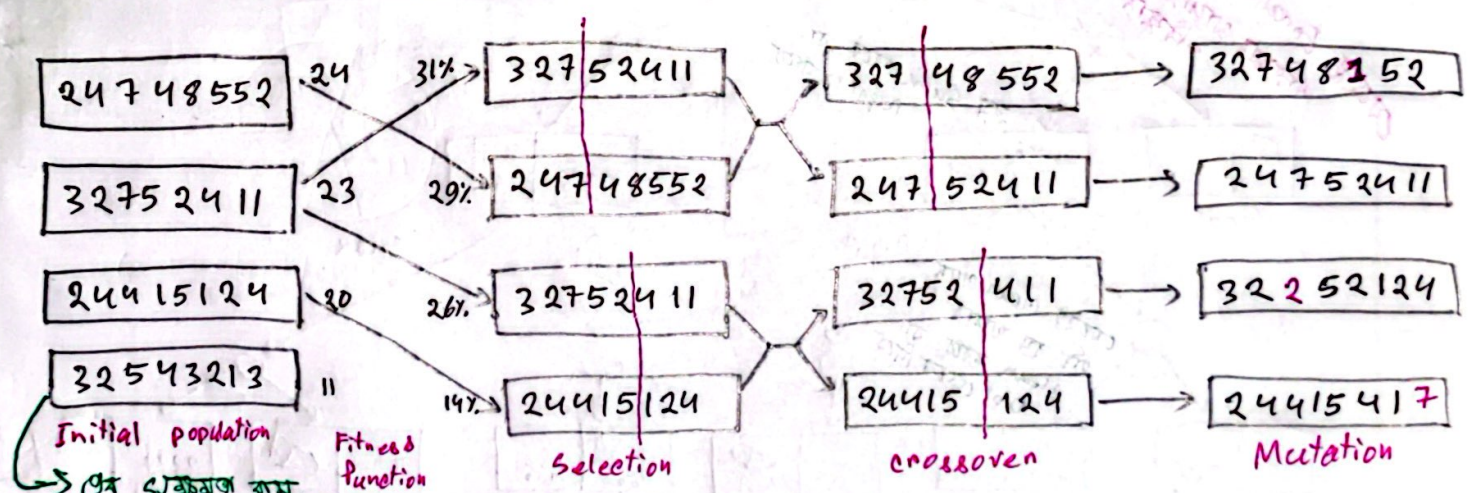
0 → যেখানে 0 আছে সেখানে অন্যের জায়গা থাকবে



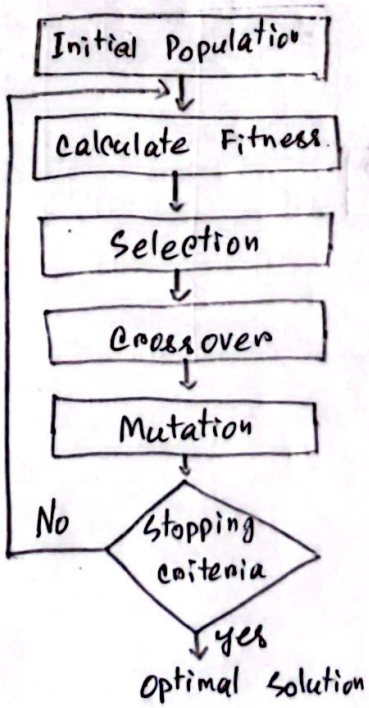
Scanned with CamScanner

Genetic Algorithms





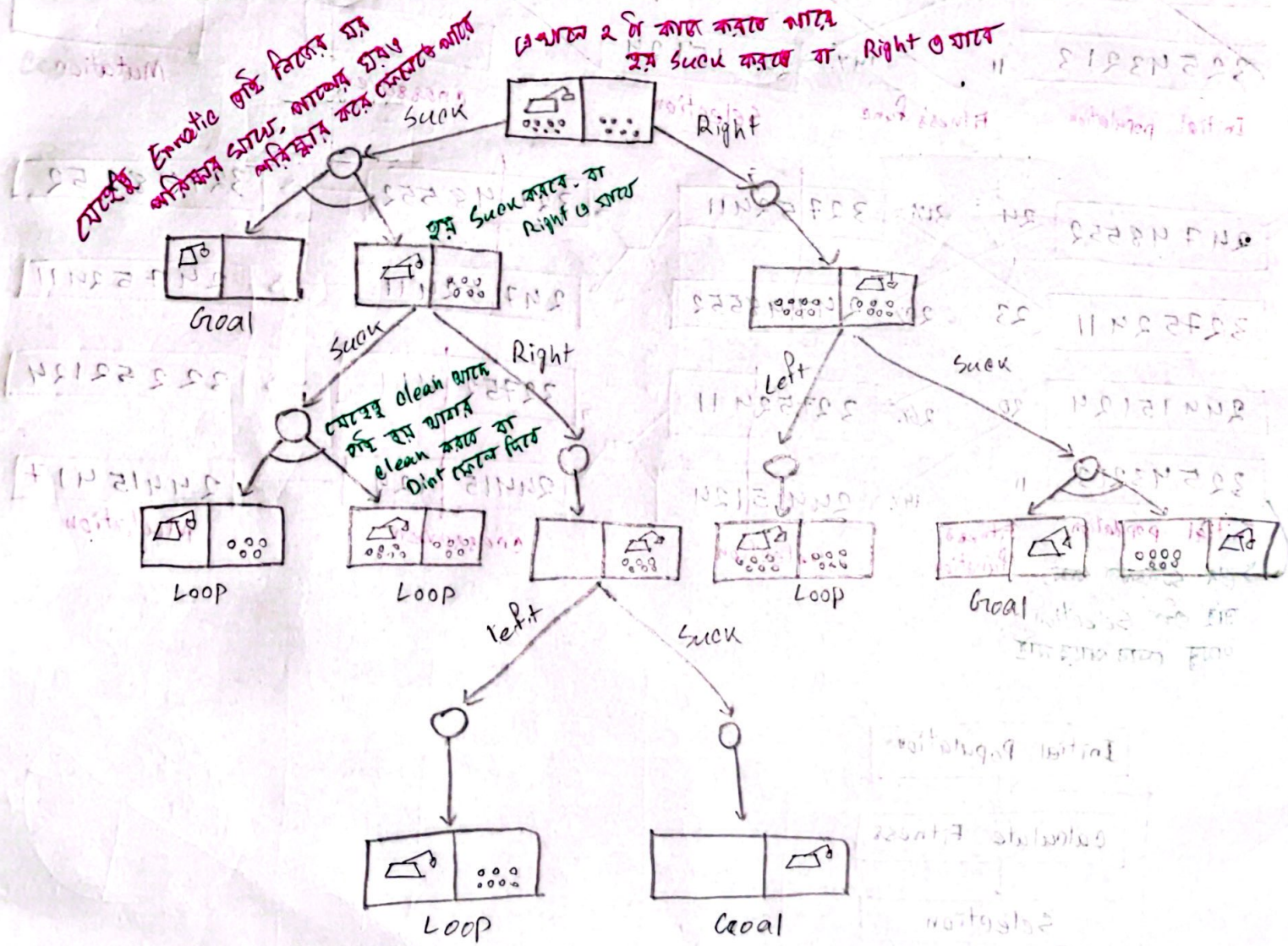
এর সমাধান করা,
তাই এর Selection
পর্যন্ত যেতে পারে নাই



Flow-chart of Genetic Algo

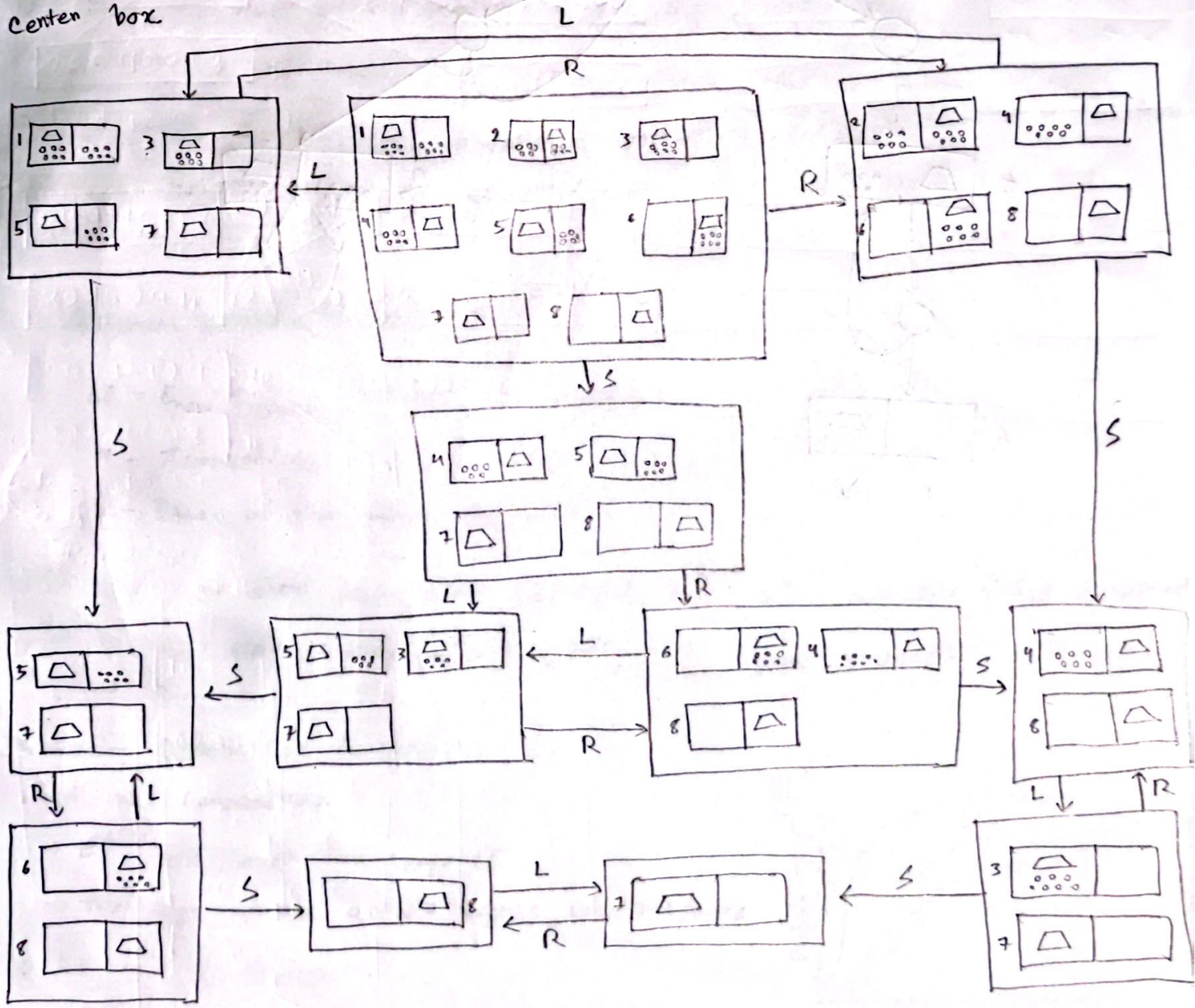
MID

In the Erratic Vaccum World, the environment has 3 action: Right, left, suck. The Suck action cleans a dirty square and may clean an adjacent square, but if applied to clean square it may desposit dirt. The goal is to clean all dirt in the environment starting from a given initial state. Find a sequence of actions leading to the goal state, draw an AND-OR search tree.



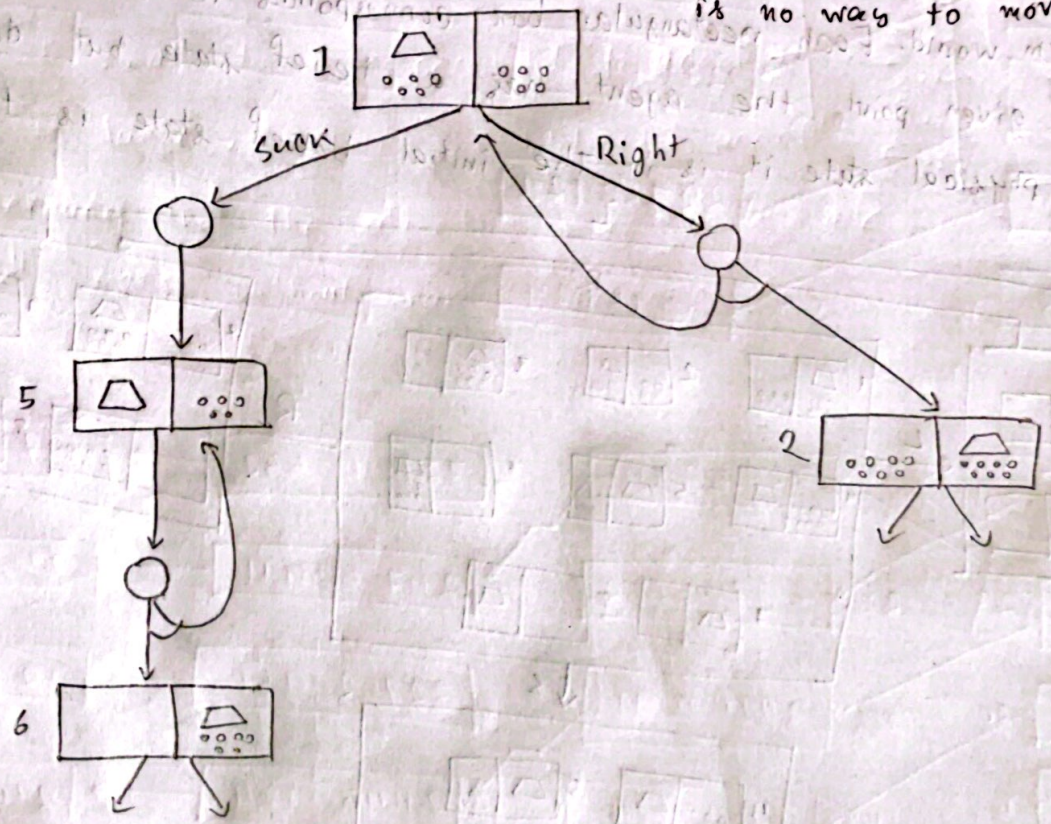
Searching with no observation! A deterministic, sensorless vacuum world

The reachable portion of the belief-state space for the deterministic, sensorless vacuum world. Each rectangular box corresponds to a single belief state. At any given point the agent has a belief state but does not know which physical state it is in. The initial belief state is the top center box



Try, try again / The Slippery Vaccum World

Part of the search graph for a slippery vaccum world where we have shown (some) cycles explicitly. All solutions for this problem are cyclic plans because there is no way to move reliably.



Mid

(6b) Define and explain the acceptance probability function in simulated annealing. Why does it depend on temperature and energy difference.

Simulated Annealing is a probabilistic optimization algorithm / technique used to find a near-optimal solution in a large and complex search space using annealing process.

In simulated Annealing the acceptance probability function determines whether a new solution should be accepted - even if it is worse than the current one.

Boltzmann Distribution, $P(\text{accept}) = e^{-\Delta E/T}$

$\Delta E = E_{new} - E_{current}$; Change in energy

T - Temperature

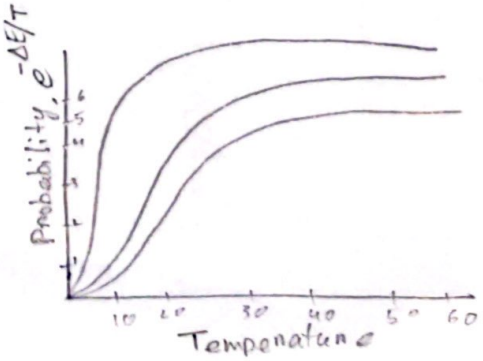
e - Base of the natural logarithm

→ If the new state is better ($\Delta E < 0$); $e^{-\Delta E/T} > 1$; new state always accepted

→ If the new state is worse ($\Delta E > 0$); $e^{-\Delta E/T} < 1$; new state is accepted with some probability

Annealing Depends on temperature

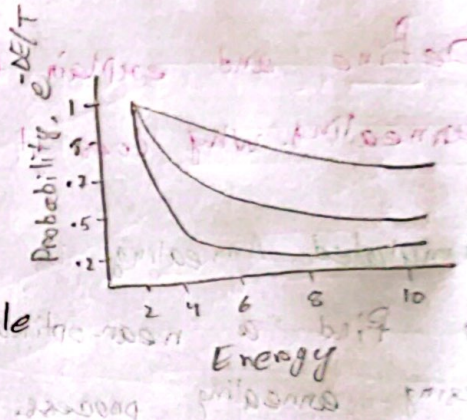
1. At high temperature
   → $e^{-\Delta E/T} \approx 1$, even for large ΔE
   → The algo always accepts worse solutions / more
2. At low temperature
   → $e^{-\Delta E/T}$ become very small for large ΔE
   → The algo rarely accepts worse solution / more



Scanned with CamScanner

Depend on Energy Difference

- ΔE measures how much worse the solution is
- If ΔE is small, it's more possibility to accepted
- If ΔE is large, it's less possibility to accept
- This ensures that the algorithm makes reasonable downhill moves without random wandering.



F

(1a) What is Artificial Intelligence (AI) & Describe 2 major applications of AI in healthcare.

Artifical Intelligence is a branch of computer science that enables machines to think, learn and make decisions like humans.

1. Disease ~~Diagn~~ Diagonosis and Prediction

AI system can analyze medical data such as X-rays, CT scans and MRI images to detect or predict diseases like cancer, pneumonia, brain tumors more accurately and faster than humans.

Example: Goodle's DeepMind AI helps ~~do~~ doctors identify eye diseases

2. Drug Discovery and Development

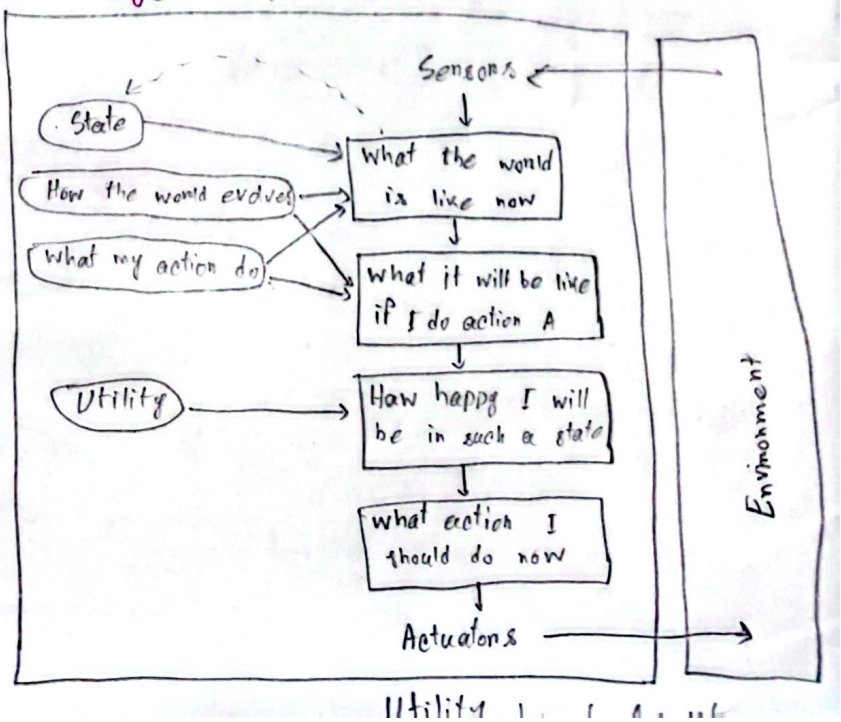
AI accelerates the process of discovering new drugs by predicting how different chemical compounds will interact with diseases

Ex: AI was used during COVID-19 pandemic to help identify drugs quickly

(2) Describe simple reflex agents and ~~g~~ goal-based agents. Provide example to illustrate how each type of agent operates in real-world scenarios.

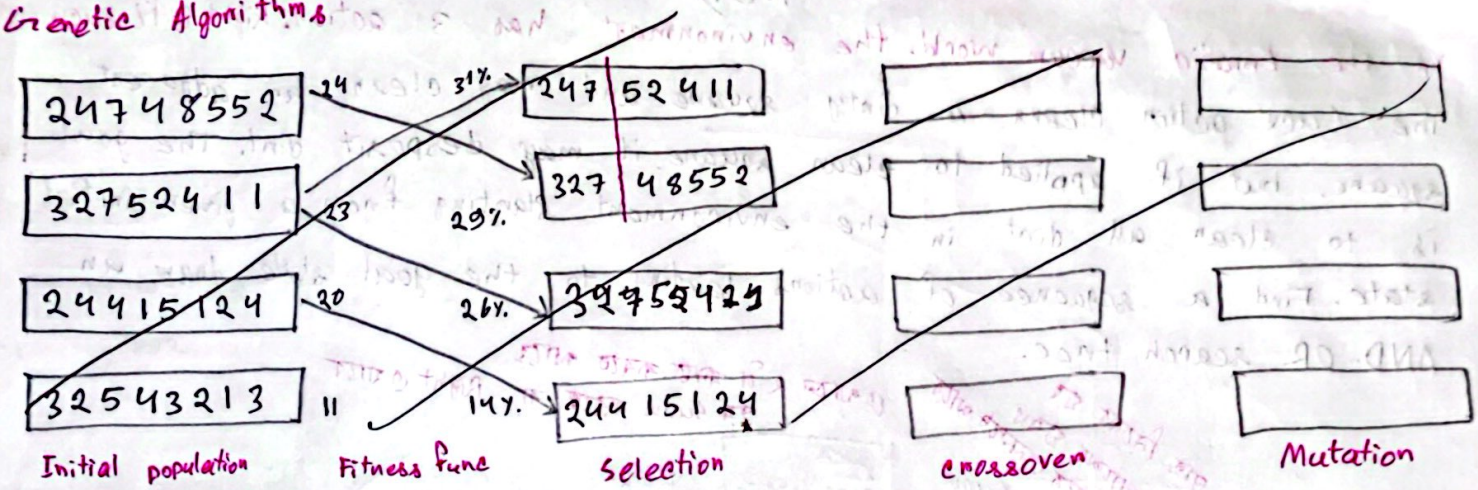
4 kind of Agent Programs

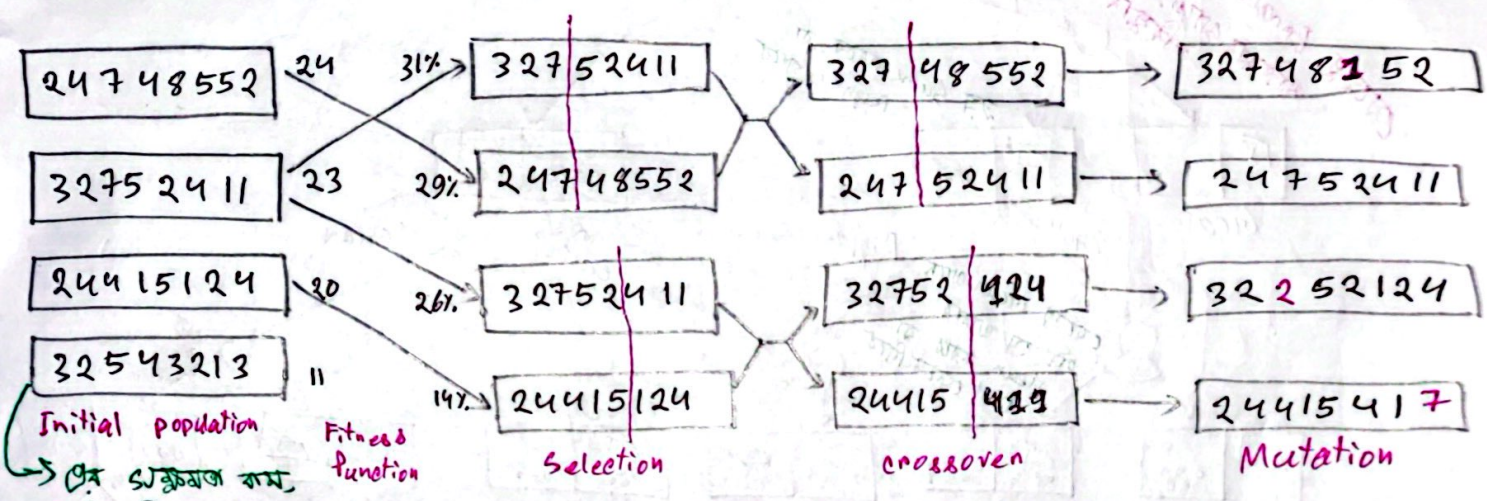
I. Simple reflex agents
II. Model-based reflex "
III. Goal-based "
IV. Utility-based "



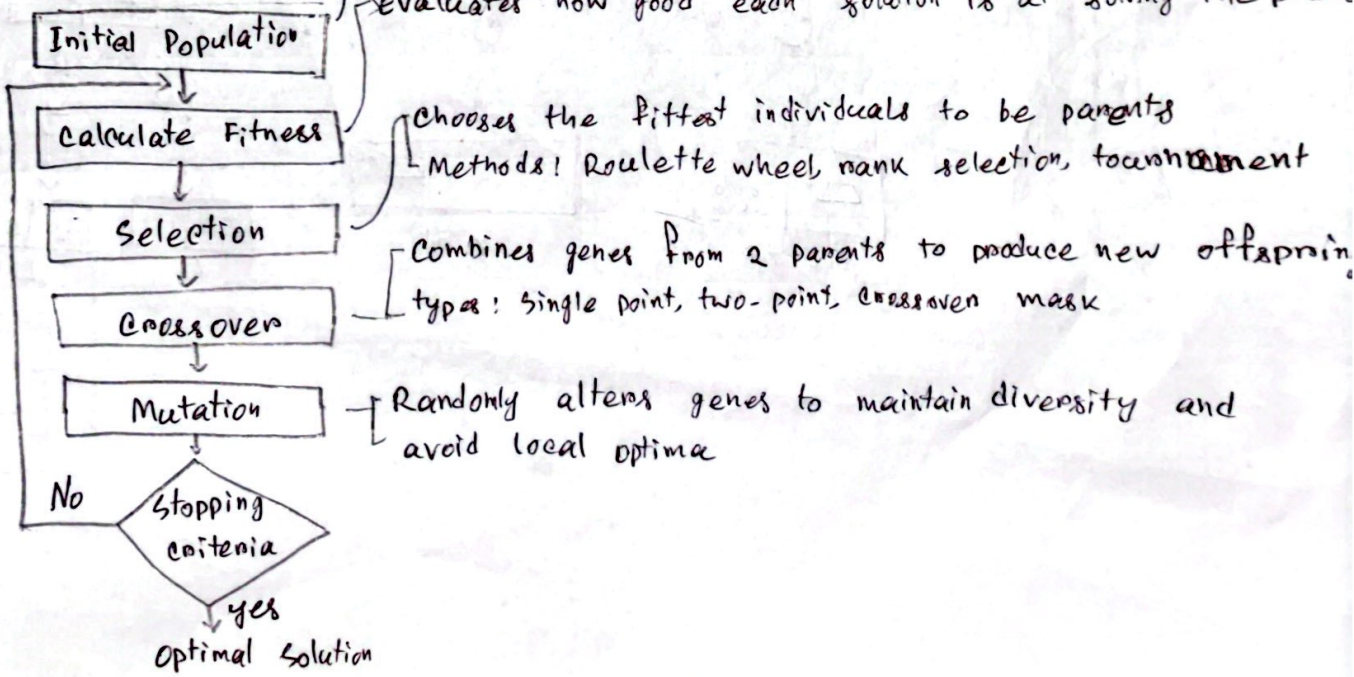
Utility

Genetic Algorithms





এর সর্বোত্তম কম্বো
পাই এবং Selection
পর্যন্ত সেরা থাকে যাই



Flow-chart of Genetic Algo

F

(1a) What is Artificial Intelligence (AI)? Describe 2 major applications of AI in healthcare.

Artifical Intelligence is a branch of computer science that enables machines to think, learn and make decisions like humans.

1. Disease ~~Diagoni~~. Diagonosis and Prediction

AI system can analyze medical data such as X-rays, CT scans and MRI images to detect or predict diseases like cancer, pneumonia, brain tumors more accurately and faster than humans.

Example: Goodle's DeepMind AI helps ~~to~~ doctors identify eye diseases

2. Drug Discovery and Development

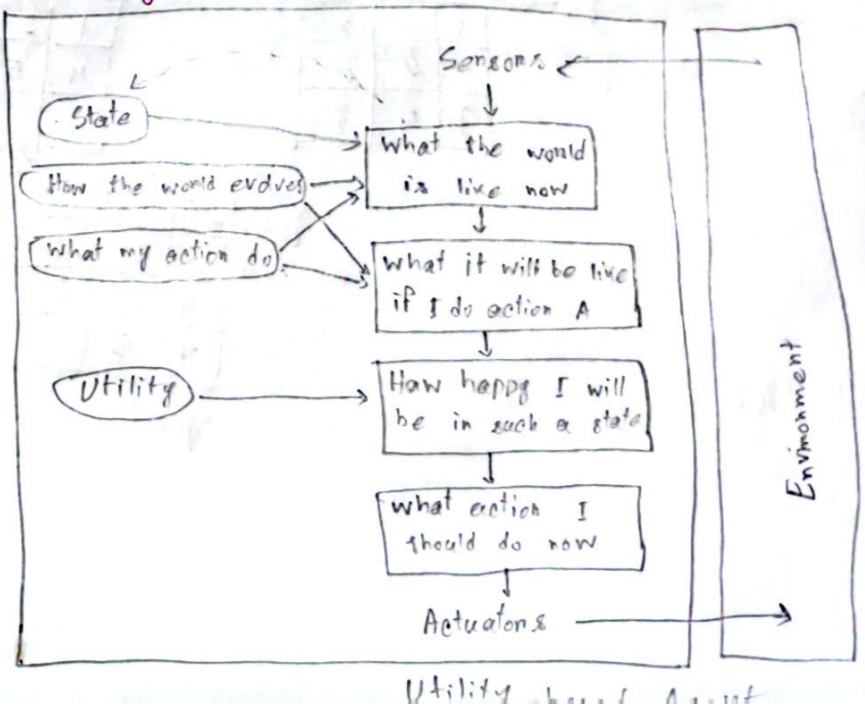
AI accelerates the process of discovering new drugs by predicting how different chemical compounds will interact with diseases

Ex: AI was used during COVID-19 pandemic to help identify drugs quicly

(2) Describe simple reflex agents and ~~g~~ goal-based agents. Provide example to illustrate how each type of agent operates in real-world scenarios.

4 kind of Agent Programs

1. Simple reflex agents
2. Model-based reflex "
3. Goal-based "
4. Utility-based "



Utility-based Agent

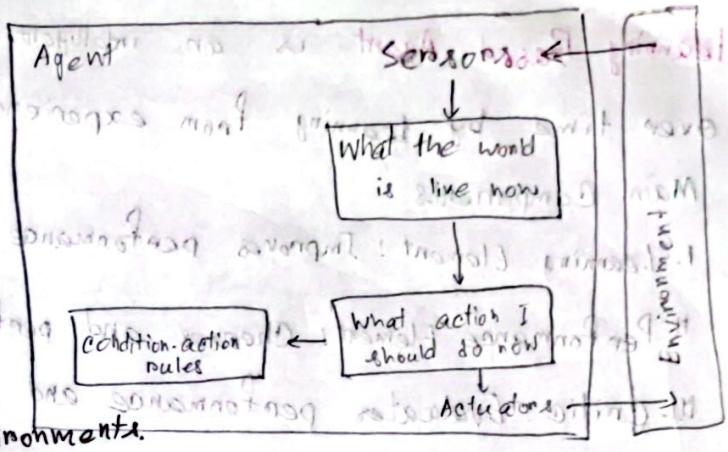
Simple Reflex Agent

- Acts only on current percept
- Uses condition-action rules (if-then)
- No memory of past percept

Exam: Vaccum cleaner: if dirty → suck.

Adv: simple and fast

Dis: Fails in partially observable environments.



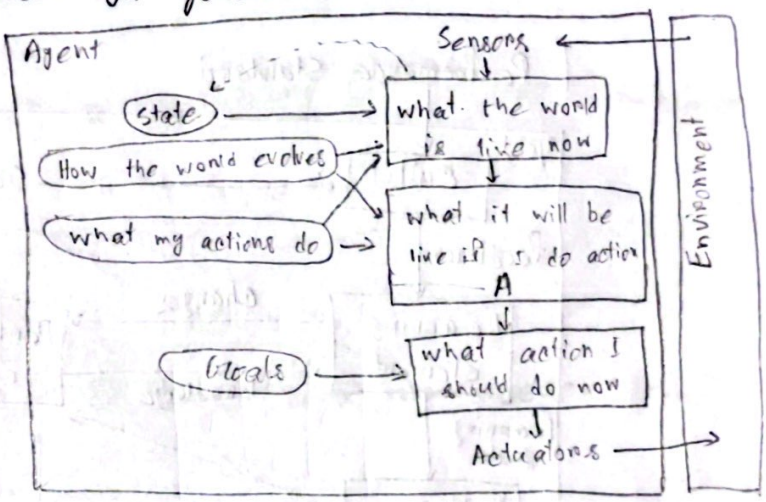
Goal Based Agent

- A goal-based agent chooses actions based on goals it wants to achieve
- Has specific goals to achieve
- Uses search and planning to choose actions
- Choose action that lead to the goal

Ex: GPS system finding shortest route

adv: Flexible for different goals

Dis: Planning can be time consuming



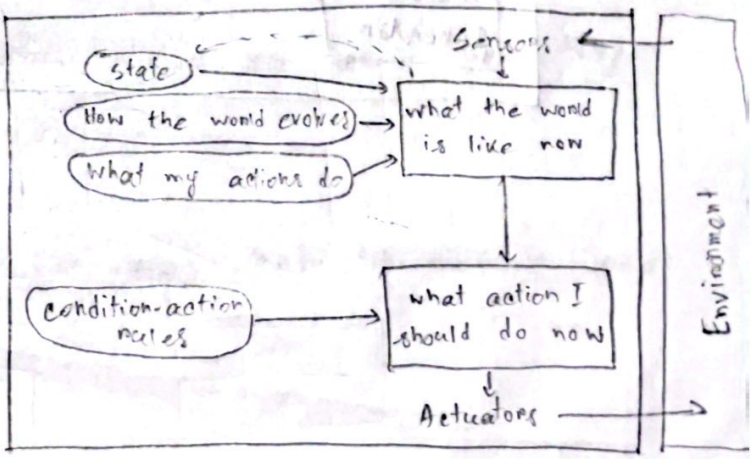
Model-based Reflex Agent

- Maintains an internal model of the world
- Uses past percepts + current percepts to decide actions
- Can handle partially observable environment

Ex: Robot vacuum with memory of room layout

adv: Smarter and more accurate decisions

Dis: Complex to design and update model



Utility Based Agent

- Uses a utility function to measure how good each state is
- chooses action that maximizes expected utility
- Balances multiple goals and preferences

Ex: Self driving can choosing safest, fastest route

adv: Produces optimal and balanced behavior

Dis: Hard to design utility function

learning Based Agent is an intelligent agent that improves its performance over time by learning from experience and feedback from the environment

Main Components

I. Learning Element : Improves performance using feedback

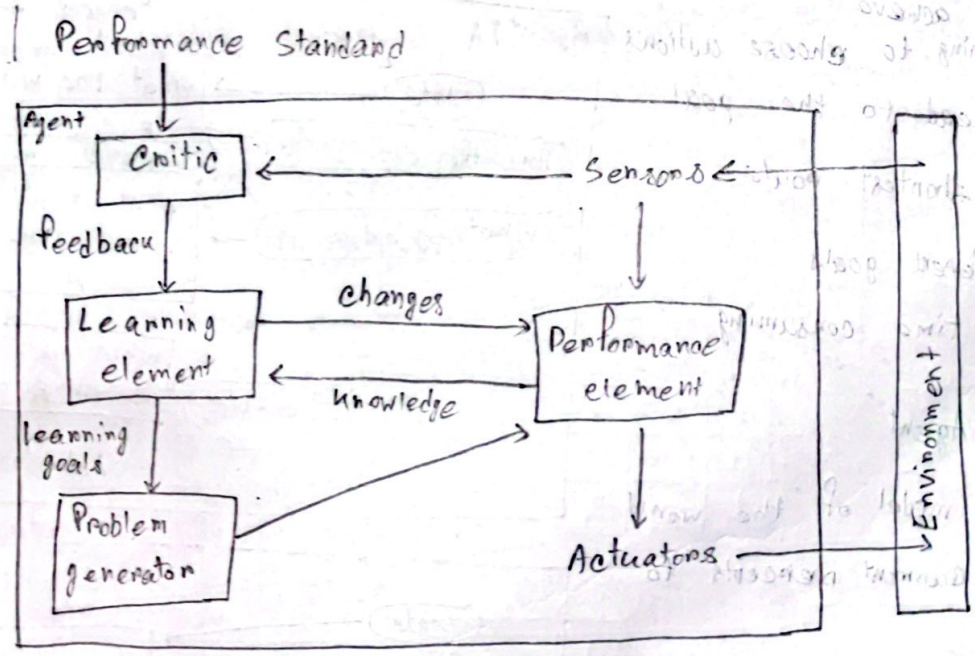
II. Performance Element : Chooses and performs actions

III. Critic : Evaluates performance and gives feedback

IV. Problem Generator : Suggests new actions for exploration

Example : Robot improving path efficiency over time

adv: Adapts to new situations automatically



(4b) Explain the concepts of local minima in the context of Hill climbing. Discuss strategies or techniques to handle local minima in Hill climbing.

In Hill climbing, a local minimum is a point in the search space where the algo gets stuck because all neighboring state are worse then current state

Stategies to Handle local Minima

1. Random-Restart Hill climbing
   - run hill climbing multiple times from different random starting points
   - Increases the chance of finding the global optimum.

2. Simulated Annealing
   - Accepting worse solutions based on a probability ($e^{-\Delta E/T}$) that decreases over time.
   - Helps ecape local minima early in the search

3. Stochastic Hill climbing
   - Randomly selects among uphill moves, may help to escape local minima.

4. Use of Memory
   - keeps a list of recently visited states to avoid revisiting them and getting stuck in loops.

5. Adding Random Noise
   - Adding noise to the current state may help to escape local minima

Scanned with CamScanner